

A Modular Open-Source Digital Model Architecture for Bioprocess Design: Validation through Stochastic Simulation of an Isolated Soy Protein Plant

Abstract ID: 1234

Author information is purposely removed for blind review

Abstract

There is growing interest in digital twins for bioprocesses. However, most frameworks work at the shadow or twin level and need IoT infrastructure, which is not available during early design. This paper presents a modular, open-source digital model architecture at Kritzinger's "Digital Model" level. The system has four decoupled Python modules: a phenomenological kernel with mass and energy balances for each unit operation, a constraint engine that checks equipment capacity limits, a configuration layer with automatic range validation, and a real-time visualization dashboard. The architecture is validated through an Isolated Soy Protein (ISP) plant case study at 1,000 kg/h. A Monte Carlo campaign of 100,000 runs on the initial design gave only 4.70% success. Based on a bottleneck analysis, equipment was resized. A second campaign of 300,000 runs achieved 99.88% success (3.03σ), with 19.5% energy savings from reverse osmosis pre-concentration. All eleven unit tests pass, which confirms numerical integrity.

Keywords

Digital twin, open-source simulation, Monte Carlo, bioprocess design, Python

1. Introduction

Kritzinger et al. [1] define three levels of virtual integration: **Digital Model** (static, no automatic data exchange), **Digital Shadow** (one-way data flow from process to model), and **Digital Twin** (two-way data flow). Most reported work focuses on the upper two levels, but these depend on commercial data platforms that make independent replication difficult [2]. There is a need for open-source platforms at the base level: the phenomenological Digital Model.

This work fills that need by presenting a modular, replicable Digital Model architecture in pure Python. It is at Kritzinger's first level: it does not need real-time data or IoT infrastructure. It can serve as a mathematically verifiable basis for future Digital Shadow or Twin development. The core uses conservation equations, not black-box models, which makes the model interpretable and adaptable to different processes [3].

The paper is organized as follows. Section 2 reviews related work. Section 3 describes the architecture. Section 4 covers replicability. Section 5 presents Monte Carlo validation on an ISP plant. Section 6 concludes and discusses future work.

2. Related Work

Several open-source digital twin frameworks have been developed. OpenTwins [4] provides compositional DT development with Eclipse Ditto and Kubernetes. CoFMPy [5] enables FMI-based co-simulation with AI. DT-Forge [6] offers a six-layer architecture for real-time telemetry. UniTwin [7] provides containerized universal DTs. TumorTwin [8] targets personalized oncology with modular Python. In bioprocessing, Udugama et al. [9] propose multi-layer models for food processing, Gamero et al. [10] introduce Lean Digital Twins for biorefineries, and Verboven et al. [11] lay the conceptual foundations for food processing DTs.

Table 1 compares these frameworks with our proposal. Most of them do not have statistical validation or automated unit testing. Our architecture is the only Digital Model that includes all four validation layers: phenomenological models, automated unit testing, stochastic analysis, and a real-world case study.

3. Methodology

The digital model development followed an iterative four-stage process:

Table 1: Comparison of digital twin frameworks.

Framework	Pure Python	Stat. Valid.	Unit Tests	Phenom. Model	Integr. Level	UI	Real Case
This work	✓	✓	✓	✓	Model	✓	✓
OpenTwins	×	×	×	×	Twin	✓	✓
CoFMPy	✓	×	×	FMU ext.	Shadow	GUI	✓
DT-Forge	✓	×	×	Optional	Twin	×	×
UniTwin	×	×	×	×	Twin	✓	×
TumorTwin	✓	×	×	✓	Model	×	✓

1. **Phenomenological formulation:** Mass and energy balances were derived from first principles (mass conservation and the first law of thermodynamics) for each unit operation in the ISP process. Nominal efficiencies were obtained from the specialized literature [12, 13].
2. **Modular implementation in Python:** Each stage was programmed as an independent function (`calc_stage_N`) that receives control variables and equipment parameters, and returns a dictionary with balance results. The function `run_process_model` connects the stages in sequence.
3. **Multi-layer verification and validation:** Three levels were implemented: (i) validation of physical ranges in input parameters, (ii) eleven unit tests that verify numerical consistency, and (iii) verification of global mass balance closure (error < 0.1%).
4. **Stochastic robustness analysis:** Monte Carlo campaigns were executed to evaluate system behavior when all 19 control variables vary simultaneously. Bottlenecks were identified through failure frequency analysis.

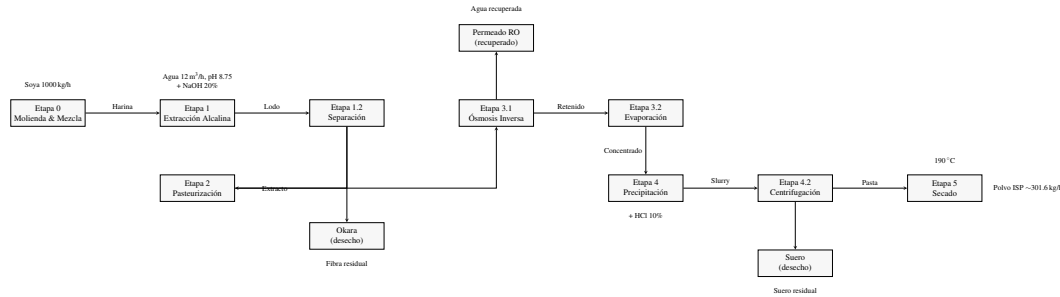


Figure 1: Process flow diagram of the ISP plant with the nine main stages.

4. Architecture

The system has four decoupled computational modules in the `core/` package, all in Python 3.10+. The modules communicate using structured dictionaries, which allows replacing one module without affecting the others.

4.1 Module Overview

- `equipment_specs.py` — Equipment configuration. Defines `EQUIPMENT_SPEC_DEFAULTS` (nominal capacities), `EQUIPMENT_SPEC_LIMITS` (physical ranges), and `CAPACITY_LIMIT_DEFAULTS` (safety factors). Automatic validation via `validate_equipment_specs` and `validate_capacity_limits`.
- `stage_equations.py` — Phenomenological kernel. Independent `calc_stage_N` functions solve mass and energy balances for each stage. `run_process_model` orchestrates the sequence, validates numerical results, computes the global mass balance closure, and calls the constraint engine.
- `equipment_constraints.py` — Constraint engine. `evaluate_capacity_constraints` checks the load against equipment capacity adjusted by safety factors. `enforce_capacity_constraints` raises the error `EquipmentCapacityError` when a limit is exceeded.

- `process_model.py` — History support. Provides `ControlLog` (circular buffer with 1,000 points) and `build_snapshot` for organizing controls, specs, and results.

The control room interface (`app.py`) uses Streamlit to integrate a simulation loop with configurable step size, first-order process inertia, and real-time KPI visualization.

4.2 Phenomenological Model

Each stage solves algebraic mass and energy balances. The equations below show the ideal balances; the implementation incorporates penalty functions for off-nominal conditions, an operational loss factor ($F_{\text{Loss}} = 2\%$ per stage), and empirical efficiency maps.

The grinding and mixing stage combines soy and water:

$$\dot{m}_0 = \dot{m}_{\text{soya}} + \rho_w Q_w \quad (1)$$

with $\dot{m}_{\text{soya}} = 1,000 \text{ kg/h}$ and $\rho_w = 1,000 \text{ kg/m}^3$.

Alkaline extraction solubilizes protein. Extraction efficiency η_{ext} is modulated by pH, temperature, residence time, and solid-liquid ratio penalties:

$$\dot{m}_{\text{prot}} = \eta_{\text{ext}} X_{\text{prot}} \dot{m}_{\text{soya}} \quad (2)$$

$$\eta_{\text{ext}} = \eta_{\text{base}} \cdot f_{\text{pH}} \cdot f_T \cdot f_t \cdot f_r \quad (3)$$

where $X_{\text{prot}} = 0.375$, $\eta_{\text{base}} = 0.88$, and each f_i is a continuous penalty function that degrades efficiency outside the optimal window (pH 8.0–9.5, temperature 45–75 °C, residence ≥ 60 min, ratio 9–14).

Reverse osmosis pre-concentration follows a simplified transmembrane pressure model:

$$\dot{m}_{\text{perm}} = \dot{m}_{\text{perm,ref}} \cdot \frac{\dot{m}_{\text{soya}}}{\dot{m}_{\text{soya,ref}}} \cdot \min \left(\frac{P}{P_{\text{ref}}}, 1.5 \right) \quad (4)$$

Isoelectric precipitation at pH 4.5 and spray drying complete the sequence. The temporal response to setpoint changes uses a first-order filter:

$$y[k+1] = y[k] + (u[k] - y[k])(1 - e^{-\Delta t/\tau}) \quad (5)$$

where τ is the time constant per variable, enabling heterogeneous dynamics across stages.

4.3 Constraint Engine

Capacity constraints compare process load against equipment capacity, adjusted by safety factors f_s :

$$V_{\text{TK}} \leq V_{\text{TK,max}} \cdot f_{s,\text{TK}} \quad (6)$$

$$\dot{Q}_{\text{HX}} \leq A_{\text{HX}} \cdot U \cdot \Delta T_{\text{lm}} \cdot f_{s,\text{HX}} \quad (7)$$

$$\dot{m}_{\text{vap}} \leq \dot{m}_{\text{vap,max}} \cdot f_{s,\text{EV}} \quad (8)$$

When a violation occurs, the system raises `EquipmentCapacityError` with a structured alarm that includes the equipment code, the load magnitude, and a recommendation for correction.

5. Replicability Requirements

Adapting the architecture to a new process requires three steps: (1) map the equipment and create `EQUIPMENT_SPEC_DEFAULTS` with design capacities, (2) implement the function `stage_n(controls, specs)` for each unit operation, and (3) define `CONTROL_LIMITS` with the operational ranges. The validation functions make sure all parameters stay within physically possible values.

6. Validation: ISP Plant Case Study

The architecture was applied to an Isolated Soy Protein (ISP) plant at 1,000 kg/h nominal capacity [12, 13]. The process has nine stages: grinding and mixing (E0), alkaline extraction (E1), primary separation (E1.2), HTST pasteurization (E2), reverse osmosis pre-concentration (RO), vacuum evaporation (E3), isoelectric precipitation (E4), centrifugation (E4.2), and spray drying (E5).

6.1 Baseline Validation

With nominal parameters ($\dot{m}_{\text{soya}} = 1,000 \text{ kg/h}$, $Q_{\text{water}} = 12 \text{ m}^3/\text{h}$, pH 8.75, $T_{\text{pasteur}} = 80 \text{ }^\circ\text{C}$, $\text{pH}_{\text{precip}} = 4.5$), the model produces:

- Extraction efficiency: 88.0%
- Protein precipitated: 292.3 kg/h
- Final powder: 301.6 kg/h (286.5 kg/h protein)
- Global protein yield: 76.4%
- Mass balance closure error: <0.1%

All eleven unit tests pass, confirming numerical integrity.

6.2 Stochastic Diagnosis

A Monte Carlo campaign of 10 batches $\times$ 10,000 runs (100,000 total) was carried out on the initial equipment design. All 19 control variables were randomly varied within wide ranges. Only 4.70% of the runs were successful ($\sigma = 0.18\%$, 95% CI: [4.59%, 4.81%]). The main bottlenecks were: extraction tank TK-101 (78.23% of failures), water tank TK-100 (69.50%), evaporator EV-301 (57.06%), and heat exchanger HX-201 (40.69%).

6.3 Optimized Design

Based on the diagnosis, the bottleneck equipment was resized (Table 2). A second campaign of 30 batches $\times$ 10,000 runs (300,000 total) with wider operational ranges ($\pm 20\%$ for feed rate, $\pm 25\%$ for water flow) gave the following results:

- **Success rate:** 99.88% (299,633/300,000)
- **Std. deviation:** $\pm 0.026\%$, 95% CI: [99.87%, 99.89%]
- **Sigma level:** $Z = 3.03\sigma$ ($\sim 1,200$ DPMO)
- **Residual failures:** Only `STAGE1_TANK_CAPACITY` in 0.12%

Table 2: Optimized capacity limits.

Equipment	Variable	Limit
TK-100	Geometric capacity	22.0 m ³
TK-101	Geometric capacity	35.0 m ³
HX-201	Heat exchange area	68.5 m ²
EV-301	Evaporative capacity	16.0 m ³ /h
CF-401	Hydraulic flow	10.0 m ³ /h
SD-501	Evaporative capacity	1,000 kg/h

6.4 Energy Analysis

Reverse osmosis removes 2,718.3 kg/h of water in liquid phase using only 8.5 kW of electrical power. This reduces the evaporator thermal load by 836.2 kW (19.5%). The net savings of 827.7 kW allow the membrane investment to be recovered in less than 1.5 years.

Table 3: Energy comparison: without RO vs with RO.

Variable	Without RO	With RO
Water removed by evaporation (kg/h)	6,841.0	5,509.0
Water removed by RO (kg/h)	0.0	2,718.3
Evaporator thermal power (kW)	4,294.6	3,458.4
Thermal savings (kW)	0.0	836.2 (19.5%)
RO electrical power (kW)	0.0	8.5
Net savings (kW)	0.0	827.7

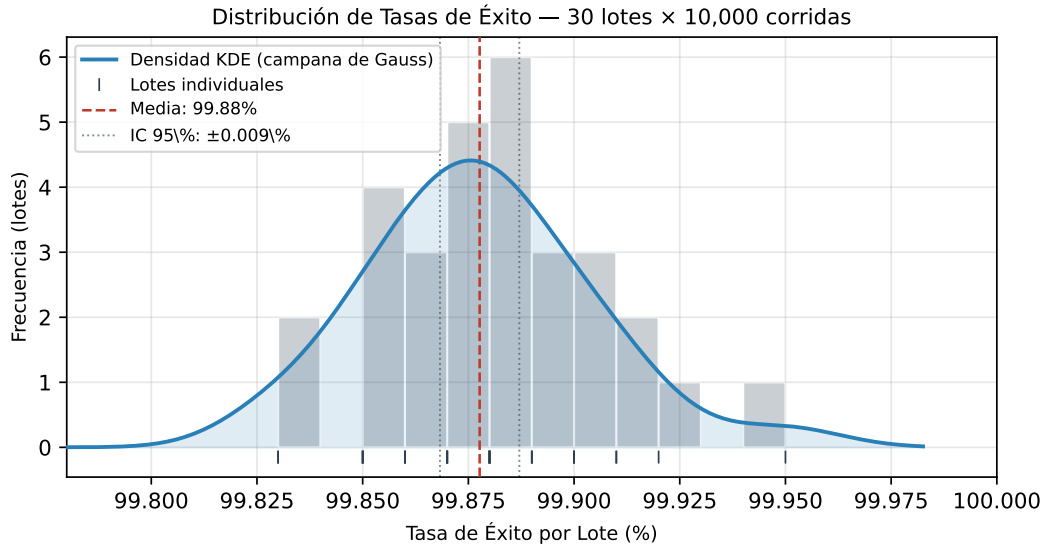


Figure 2: Distribution of success rates per batch for the optimized campaign (30 batches of 10,000 runs). The KDE curve approximates a Gaussian; the dashed red line marks the mean (99.88%); dashed gray lines delimit the 95% CI ($\pm 0.026\%$).

7. Conclusion and Future Work

We presented a modular, open-source Digital Model architecture validated through Monte Carlo simulation of an ISP plant. The key features that make it replicable are: externalized configuration with automatic validation, independent and replaceable stage equations, a decoupled constraint engine, and automated unit testing.

The stochastic diagnosis improved the design from 4.70% to 99.88% success (3.03σ), and the energy analysis showed a 19.5% savings from RO integration. These results show that the architecture can be used for systematic process diagnosis and optimization during the conceptual design phase.

The source code is publicly available under MIT license, including the phenomenological kernel, constraint engine, and 11 unit tests. The Monte Carlo experiments can be reproduced with `python scripts/plot_monte_carlo.py`. Future work includes: IoT integration for Digital Shadow transition, multi-plant composition using the OpenTwins approach, and hybrid AI models for product quality prediction.

Acknowledgements

Acknowledgements are purposely removed for blind review.

References

- [1] W. Kritzinger *et al.*, “Digital twin in manufacturing: A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [2] S. Gil *et al.*, “Survey on open-source digital twin frameworks,” *Software Practice and Experience*, vol. 54, no. 6, pp. 929–960, 2024.
- [3] R. Perry and D. Green, *Perry’s Chemical Engineers’ Handbook*. McGraw-Hill, 9th ed., 2019.
- [4] J. Robles, C. Martin, and M. Diaz, “Opentwins: An open-source framework for compositional digital twins,” *Computers in Industry*, vol. 152, p. 104007, 2023.
- [5] C. Friedrich *et al.*, “Cofmpy: A python framework for fmi-based digital twins,” in *Proc. 2nd Int. Conf. Engineering Digital Twins*, 2025.
- [6] D.-F. Contributors, “Dt-forge: Domain-agnostic python framework for digital twin architectures,” 2025.
- [7] T. Haußermann *et al.*, “Unitwin: Universal digital twins through reconfigurable container environments,” *IEEE Internet Computing*, vol. 29, no. 1, pp. 8–15, 2024.
- [8] T. Contributors, “Tumortwin: A python framework for patient-specific digital twins in oncology,” 2025. arXiv:2505.00670.
- [9] I. Udugama *et al.*, “Digital twins in food processing: Multi-layer digital models,” *Digital Chemical Engineering*, vol. 6, p. 100087, 2023.
- [10] E. Gamero *et al.*, “Data management in biorefineries: Lean digital twinning,” *Procedia CIRP*, 2024.
- [11] P. Verboven *et al.*, “Digital twins of food process operations,” *Current Opinion in Food Science*, vol. 35, pp. 79–87, 2020.
- [12] E. Lusas and M. Riaz, “Soy protein products: processing and use,” *The Journal of Nutrition*, vol. 125, no. suppl 3, pp. 573S–580S, 1995.
- [13] J. Kinsella, “Functional properties of soy proteins,” *JAOCs*, vol. 56, pp. 242–258, 1979.